

Moral-Continuity Ledger (MCL) — rc1.1 Spec & Research Protocol

Scope. This document consolidates the CCN v2 → MCL design into a research-grade, auditable package with a policy pack, reproducible CLI, and classification-sensitivity instrumentation. It's written to be printed, shared, and iterated on.

0) One-page summary

- **Three tracks (engineering split):**
- **CM (Continuity Mass)** — conserved, root-attached stake; redistributes over continuers via shares $s(i, \cdot)$.
- **CR (Creation Mass)** — new moral stake; **split** into **CR_true** (physics creation) and **CR_reclass** (policy-induced reclassification).
- **CH (Centerhood)** — descriptive multiplicity counter; policy-visible (can be demoted/omitted).
- **Core representation (finite branching):** $\mathcal{V}(w) = \underbrace{\sum_{i \in R_w} \omega(i) \sum_{h \in H_i} s(i, h) F(u(h))}_{\text{CM}(w)} +$

$$\underbrace{\sum_{c \in C_w} \omega(c) G(u(c))}_{\text{CR}(w)}.$$

With **branch-mixture linearity** + **monotonicity** + regularity, CM has an expected-utility form at the root level. Replication invariance follows: duplicating branches preserves CM.

- **Instrumentation:** audits for invariances, transform hygiene, classification sensitivity; **policy pack** for side-by-side comparisons; **CLI** for JSON-in/JSON-out reproducibility; **world diff** for explainability.
-

1) Data model (JSON)

```
{
  "id": "world-id",
  "classification_policy_id": "v0",
  "ch_mode": "external | computed",
  "centers": null,
  "metadata": {"weights_version": "...", "transforms_version": "..."},
  "roots": [
    {
      "id": "i",
      "weight": 1.0, // w(i)
      "histories": [
        {"id": "h1", "welfare": 1.0, "share": 0.4, "classification_confidence":
```

```

1.0},
  {"id": "h2", "welfare": 3.0, "share": 0.6, "classification_confidence":
1.0}
]
}
],
"created": [
  {"id": "c1", "weight": 1.0, "welfare": 2.0, "kind": "true",
"classification_confidence": 1.0}
]
}

```

Notes.

- `kind ∈ {"true", "reclass"}` splits CR into CR_{true} vs CR_{reclass} .
- `ch_mode="external"` (default) demotes CH unless `centers` is provided. `computed` counts continuers + created.

2) Transforms and defaults

- **Defaults:** `F = id`, `G = id` (identity). Prefer **F = G** unless explicitly justified.
- Safe alternatives: `F_tanh` (all reals), `G_shifted_log` (domain-guarded, monotone).
- The audit suite surfaces monotonicity, symmetry (F vs G), sensitivity (variance over transform families), and scale response.

3) Policy pack (side-by-side)

Run policies on the **same** world and compare outputs:

- **baseline** — no reclassification.
- **threshold $_{\tau}$** — if a history's `classification_confidence <  $\tau$` , route its CM share to an **outside option** branch (`welfare = outside_welfare`) and add a **reclassified** created item with weight `$\omega(\text{root}) \cdot \text{share}$` and the original welfare.
- **expected** — split each history into `c` portion (stays in CM) and `1-c` portion (goes to outside option); add a **reclassified** created item with weight `$\omega \cdot \text{share} \cdot (1-c)$` and the original welfare.
- **outside-option family** — like `threshold`, but with explicit outside welfare (e.g., `outside_-1@0.5`).

Invariant. Reclassification **moves** stake; it does **not** mint stake. With `F = G`, total is invariant under these policies.

4) Audits (research-grade checklist)

1. **flow_sums** — per-root shares sum to 1.
 2. **mixture_linearity** — numeric spot-checks of $V_i(tp+(1-t)q) = tV_i(p)+(1-t)V_i(q)$.
 3. **replication_invariance** — fission/duplication leaves CM unchanged.
 4. **creation_modularity** — CM $\perp$ creation; CR $\perp$ branching.
 5. **permutation_invariance** — reorderings don't change values.
 6. **transform_monotonicity** — F,G increasing on encountered domain (supports M3/ordering).
 7. **transform_sensitivity** — variance of CM/CR across candidate transforms.
 8. **scale_response** — with identity F, CM scales $\propto$ welfare units.
 9. **classification_sensitivity** — randomized reclassification using `classification_confidence`; report max $|\Delta CM|$, $|\Delta CR|$.
 10. **transform_symmetry** — flag if $F \neq G$ (recommend equality unless justified).
-

5) CLI (repro microscope)

All commands are JSON-in/JSON-out.

```
# Compute values under a policy
python mcl_rc11.py compute world.json --F id --G id --policy baseline

# Full audit suite (JSON report)
python mcl_rc11.py audit world.json --policy threshold_0.7 --seed 1

# Diff two worlds under a policy
python mcl_rc11.py diff world_A.json world_B.json --policy expected

# Compare multiple policies side-by-side
python mcl_rc11.py compare-policies world.json
--policies baseline,threshold_0.5,expected --outside 0.0
```

Outside option. Use `--outside <value>` to set welfare for the policy's outside branch.

6) World diff & explainability

- `world_diff(w1,w2,F,G)` → ΔCM , ΔCR , ΔCR_{true} , $\Delta CR_{\text{reclass}}$, ΔCH , plus **per-root** and **per-created** breakdowns.
- `audit_diff(a1,a2)` → which audits flipped (pass→fail or fail→pass) between two runs.

Use these to turn philosophical disagreements into **pinpointed deltas**.

7) Conceptual hygiene (rc1.1 stance)

- **CR_true vs CR_reclass:** keeps “physics creation” distinct from “policy reclassification.” Publish them separately.
 - **CH as policy-visible:** if the locus notion is contested, keep `ch_mode="external"` and omit CH from primary outputs.
 - **Transforms are normative:** default $F = G = \text{id}$; if you deviate, justify and expect sensitivity to show up in audits.
-

8) Deliverables (local package)

- Reference implementation: `mcl_rc11.py`
- Demo notebook: `mcl_demo_rc11.ipynb`
- One-page spec (this doc is the longform): `mcl_ledger_spec_rc11.md`

Local paths (for convenience):

- `/mnt/data/mcl_rc11.py`
 - `/mnt/data/mcl_demo_rc11.ipynb`
 - `/mnt/data/mcl_ledger_spec_rc11.md`
-

9) Minimal worked example (toy)

One root `i` with two continuers `(0.4@u=1, 0.6@u=3)` and one created `c` with `u=2`. With $F=G=\text{id}$:

- $\text{CM} = 0.4 \cdot 1 + 0.6 \cdot 3 = 2.2$
 - $\text{CR}_{\text{true}} = 2.0$, $\text{CR}_{\text{reclass}} = 0.0$
 - CH (computed) = 3 (2 continuers + 1 created)
 - Fissioning a branch doubles loci but preserves CM.
-

10) Governance & provenance

- **Version:** `classification_policy_id`, transforms (F,G) , welfare model u , weights ω , and flow estimator $s(i, \cdot)$.
 - **Registry:** explicit `kind` for each created item; provenance for policy-induced reclassifications.
 - **Repro:** keep CLI outputs + input JSONs under version control. FAIR-style metadata recommended.
-

11) Roadmap (rc2 ideas)

- **Benchmarks/fixtures:** archetypal worlds (pure fission, gradual replacement, fusion, borderline continuers).
 - **Policy bundles:** multiple policies run side-by-side with a single `compare` report.
 - **Outside-option catalogue:** domain-specific outside welfare priors with citations.
-

External references

(selection; for orientation and best practice alignment)

- von Neumann & Morgenstern, **Expected-utility representation & independence**. *Theory of Games and Economic Behavior*; see accessible overviews.
- Derek Parfit, **Reasons and Persons** — fission and what matters in survival.
- David Lewis, **"Survival and Identity"** — fission/fusion, overlapping persons.
- Stanford Encyclopedia of Philosophy, **Identity over Time** — survey of metaphysical issues relevant to fission/fusion.
- Wilkinson et al., **FAIR Guiding Principles** — reproducible data stewardship (Findable, Accessible, Interoperable, Reusable).
- Mitchell et al., **Model Cards for Model Reporting** — documentation/audit norms for research-grade releases.

(Direct URLs included in the companion chat message.)

Appendix A — API sketch (Python)

```
from mcl_rc11 import (
    World, Root, History, Created,
    compute_mcl, run_audits, world_diff,
    apply_policy, F_identity, G_identity,
)

w = World(
    id="toy", ch_mode="computed",
    roots=[Root(id="i", weight=1.0, histories=[
        History(id="h1", welfare=1.0, share=0.4, classification_confidence=1.0),
        History(id="h2", welfare=3.0, share=0.6, classification_confidence=1.0),
    ])],
    created=[Created(id="c", weight=1.0, welfare=2.0, kind="true")],
)

vals = compute_mcl(w, F_identity, G_identity)
print(vals)  # {"CM": 2.2, "CR_true": 2.0, "CR_reclass": 0.0, "CR": 2.0,
```

```
"total": 4.2, "CH": 3}
```

```
# Compare policies
```

```
w_exp = apply_policy(w, "expected", outside_welfare=0.0)
```

```
print(compute_mcl(w_exp))
```

Appendix B — Audit report fields

Each audit returns `{name, passed, metrics, notes}`. Capture max gaps (absolute), ratios, and booleans; store alongside inputs for reproducibility.

Citizen Gardens © 2025 CC-NC-ND 4.0